Garbage-Guard Final Defense Script

Spoken script for Garbage-Guard Final Defense v2.pptx, checked against Garbage-Guard v68.docx

Speaker order

Speaker	Slides	Part	Approx. time
King Perth G. Pepito	1 to 4	Opening, the researchers, background of the study, research gap	about 5 minutes
Jade C. Gunayan	5 to 6	Objectives, scope and limitations	about 4 minutes
Brian Jay Celeste	7	Design: how Garbage-Guard works	about 3 minutes
Rodel S. Banogbanog	8	The prototype	about 2 minutes
Genesis Walter D. Reyes II	9 to 15	Features, testing, results, conclusions, recommendations, closing	about 11 to 12 minutes
Total	15 slides		**about 25 minutes**

Times assume a calm speaking pace of about 130 words per minute. Lines in grey italics are cues and are not read aloud. Lines in blue italics are the handoff to the next speaker.

King Perth G. Pepito (Slides 1 to 4)

Slide 1: Title

Good morning to the members of the panel, to our thesis adviser, Dr. Jonel R. Macalisang, and to everyone here today.

We are here to present our thesis, Garbage-Guard: Development of Computer Vision for Creeks Garbage Detection Using Convolutional Neural Network.

In simple words, we built a camera unit that watches a creek, counts the garbage floating on the water, and warns the barangay when enough of it has piled up.

[Next slide]

Slide 2: The researchers

These are the people behind the study.

I am King Perth G. Pepito. I handled the systems analysis, the project rules and the proofreading.

Genesis Walter D. Reyes II is our first project developer. He trained the YOLO26n garbage detection model.

Brian Jay Celeste is our second project developer. He worked on the system architecture, the model training and the Android app.

Jade C. Gunayan handled the research testing and the statistical analysis of the prototype.

Rodel S. Banogbanog handled fabrication. He built and assembled the unit.

And our thesis adviser is Dr. Jonel R. Macalisang of TUP Manila.

[Next slide]

Slide 3: Garbage in urban creeks

Let us start with the problem.

The photo on the left is the creek in Barangay 870, Pandacan, Manila. This is where we tested our system.

The problem happens in four steps. First, dumping. Waste thrown into drains ends up in the creek. Second, build-up. Bottles, bags and food containers collect in places where the water slows down. Third, blockage. The pile narrows the channel and slows the current even more. And fourth, flooding. A blocked creek overflows, and it leaves behind stagnant, foul water.

This is not only a local problem. Meijer and colleagues reported in 2021 that more than one thousand rivers are responsible for 80 percent of the plastic that rivers carry into the ocean. Creeks and rivers here in the Philippines regularly receive waste from the communities around them.

Today, a barangay usually finds out about a build-up by having someone walk along the bank and look. That is slow, it takes manpower, and it only happens from time to time.

So our question was simple. Can a camera tell the barangay when a creek needs clearing?

[Next slide]

Slide 4: The research gap

Other researchers have worked on parts of this problem. On the left are the four kinds of studies we reviewed, and what each one leaves out.

Detection and tracking studies, such as Nunkhaw and Miyamoto, and Li and colleagues, both in 2024, can find floating waste in video. But they do not keep a count over time, and they do not send an alert.

Sensor-based systems, such as Te and colleagues in 2024 and Arante and colleagues in 2025, warn people when the water level rises. But they cannot tell what is actually in the water.

Waste monitoring systems, like the one by Balachandran and Talavera in 2023, measure garbage inside bins and containers, not in moving water.

And surveys and satellite studies, such as Meijer and colleagues and Victoriano and colleagues, give readings only once in a while. There is nothing a barangay can act on the same day.

None of the systems we reviewed puts the pieces together for a creek. Garbage-Guard combines four of them.

One, a detector trained on pictures of the creek itself, running on a small computer right beside the water.

Two, counting only inside an area of water that the operator draws on the screen. The bank, the bridge and the floating barrier are ignored.

Three, one identity for each item across video frames. A bottle drifting past is counted once, not again in every frame.

And four, an alert when the count reaches a set threshold, with a saved snapshot as proof. The alert shows on the screen, sounds on a buzzer, and reaches a phone.

To explain what we set out to achieve and the boundaries of the study, here is Jade Gunayan.

[Next slide]

Jade C. Gunayan (Slides 5 to 6)

Slide 5: Objectives

Thank you, King.

Our general objective was to develop a computer vision system for creek garbage detection using a convolutional neural network and YOLO26.

A convolutional neural network is a type of program that learns to recognize objects in pictures by studying many labelled examples. YOLO, short for You Only Look Once, is a family of these models built for real-time detection, so it can keep up with live video.

We broke the general objective into three specific objectives.

The first was to design the model. We trained a YOLO26n network to detect three kinds of garbage: bottles, bags and food containers. We measured it with precision, recall and mAP at 0.5, using a held-out test set, which means pictures the model never saw while it was learning.

The second was to build the prototype. This is a Raspberry Pi 5 unit that detects garbage only inside an area the operator draws, follows each item from one video frame to the next, counts the items, and raises an alert when the count reaches 20. Twenty is only the default, and it can be changed. The alert appears on a touchscreen on the unit and on an Android app.

The third was to evaluate the system using ISO/IEC 25010:2023, an international standard for software quality. We used three of its characteristics. Functional suitability asks: does it do its job correctly? Performance efficiency asks: is it fast and stable enough? And usability asks: can the people who will use it actually work with it? For this we ran our own developer tests and an end-user survey.

[Next slide]

Slide 6: Scope and limitations

This slide shows what the study covers, and what it does not.

Starting with the scope. The detector is YOLO26n, running on a Raspberry Pi 5. It recognizes three classes: bottles, bags and food containers.

Our dataset was split 70, 10 and 20. Seventy percent of the images were used for training. Ten percent were used for validation, which checks the model while it is still learning. And twenty percent were set aside as the test set. That test set was frozen, so every later version of the model was graded on the exact same pictures. Our target was a precision of at least 80 percent.

The system counts only inside the drawn area, and it alerts at 20 items, which the user can adjust. Results appear on a touchscreen dashboard and on an Android app. And the system was deployed in the field at a creek in Barangay 870, Pandacan.

Now the limitations. First, the camera can only see floating, visible waste. Items that have sunk, or are hidden under other items, are missed. Second, the model does not detect other materials such as metal, glass or organic matter. Third, the results depend on where the camera is placed and on daylight. Night operation was left out of this study. And fourth, Garbage-Guard is for monitoring and early warning only. It does not collect the garbage, and it does not test the quality of the water.

Next, Brian Celeste will walk you through how the system works.

[Next slide]

Brian Jay Celeste (Slide 7)

Slide 7: How Garbage-Guard works

Thank you, Jade.

This slide follows one video frame through the system, from left to right.

It starts with the camera, a HiWatch HWI-B140H, which sends video at 1280 by 720 pixels.

Next is the crop. The system cuts out only the area of water that the operator drew. This keeps small items like bottles and bags large enough for the model to see, and it leaves the bank out of the picture.

Then the model detects. YOLO26n is converted to a format called NCNN, which is the format that runs fastest on the Raspberry Pi, and it looks for garbage in the cropped picture at 960 pixels.

Then it tracks. A method called ByteTrack gives each item its own ID number and keeps that number from frame to frame, so the same bottle is not counted twice.

Then it counts. An item is counted when the center of its box is inside the drawn area. To keep one bad frame from causing a false alarm, the system does not use the count from a single frame. It takes the median, or the middle value, of the last 15 frames.

Then comes the decision: has the count reached the threshold? By default, that is 20 items.

If it has, the alert goes out in four ways. A banner appears on the touchscreen dashboard. A buzzer sounds and repeats until someone mutes it. The Android app gets a notification with a snapshot, sent through Firebase. And the event is saved with its snapshot in a local SQLite database, which becomes the history.

As the box on the right says, there is only one alert per build-up. Once the alert is raised, it is not raised again until the count falls back to 15. That way, the barangay does not receive the same alarm over and over for the same pile.

At the bottom is the hardware: a Raspberry Pi 5 with 8 gigabytes of memory; a Waveshare UPS HAT (E) with four 21700 battery cells, which keeps the unit running during a power cut; a 7-inch touchscreen; a 256-gigabyte SSD for storage; a passive buzzer; and a USB Wi-Fi adapter for the internet connection.

Rodel Banogbanog built this unit, and he will now show you the prototype.

[Next slide]

Rodel S. Banogbanog (Slide 8)

Slide 8: The prototype
Thank you, Brian.

On the left is the assembled unit. Everything is housed in a metal junction box that measures 25 by 30 by 15 centimeters. The 7-inch touchscreen is set into the door, and there is a case fan on each side to help keep the inside cool. The camera is mounted outside the box. Only its cable runs back to the unit.

The photo at the top right shows the inside of the box. Letter a is the back of the touchscreen, mounted on the door. Letter b is one of the fans. Letter c is the board stack: the storage board on top, the Raspberry Pi 5 with its cooler in the middle, and the UPS board underneath. Letter d is the four 21700 battery cells. Letter e is the buzzer. Letter f is the cable that carries both the network signal and the power to the camera. Letter g is the Raspberry Pi power adapter. And letter h is a step-up cable that raises 5 volts to the 12 volts the camera needs.

At the bottom right, you can see us on site. The camera was mounted on the fence above the creek. Because of the UPS, the unit keeps running even when the power goes out.

One practical note. Once we cut openings in the box for the screen, the fans and the cables, it was no longer fully weatherproof. So the box is kept under cover, and only the camera is out in the open.

Genesis Reyes will now take you through the features, our testing, and the results.

[Next slide]

Genesis Walter D. Reyes II (Slides 9 to 15)

Slide 9: Dashboard and mobile app

Thank you, Rodel.

This is what the user actually sees.

On the left is the touchscreen dashboard during an accumulation alert. Each item the model finds gets a live box showing its class, its ID number and its confidence, which is the percentage that shows how sure the model is. The outline over the water is the drawn area. The operator sets it simply by tapping points on the water. On the left edge, the count bar reads 20 of 20, which is why the alert is active.

In the middle is the phone alert. It shows the number of items on the water, the limit, and the snapshot. In this example, it reads 20 items on the water, with a limit of 20. The user can mute the alert for 15 minutes straight from the notification, or open the app.

The two screens share one record. The app shows the same alerts and counts as the touchscreen, either over the local network or through Firebase when the user is away.

Along the bottom are the main features: live boxes with class, ID and confidence; the drawn area; a history of every alert with its snapshot; and remote settings, where the user can change the threshold, control the buzzer, and even upload a new model without editing any code.

[Next slide]

Slide 10: Development and testing

Now, how we built and tested it.

We followed a V-model with Scrum sprints. In each sprint we chose one change, built it, and tested it. If the result was weak, it did not move forward. It became the reason for the next sprint.

The timeline at the top shows the ten training runs of the model. Version 1 used public data only. Version 5 was the first to include our own photos of the creek, and that is when the scores started to rise. At version 7, the test set was frozen at 807 images, so every model after that was graded on the same pictures. Version 8 moved to a detection size of 960 pixels and added frames from the unit's own camera. And version 10, our final model, added photos from our third data session at the creek.

Testing happened at three levels.

Unit testing happened every sprint. We tested each model on the frozen test set, checked the counter logic, and logged the processing time. All ten training runs were tested this way, under test cases TC_UNIT_01 to 03.

System testing covered the fully assembled unit. There were ten test cases: detection, alerting, latency, system flow, the dashboard, the drawn area, tracking, storage, alert delivery and the threshold. Each was run ten times, for 100 trials in total.

Acceptance testing was the field deployment, held on November 6, 7 and 9. We checked classification, alerting and latency against our developer indicators, and we ran the end-user survey.

[Next slide]

Slide 11: Detection model results

Here are the results of the final model on the frozen test set of 807 images.

Precision is 84.6 percent. In plain terms, when the model says something is garbage, it is right about 85 times out of 100. That is in the Good band and above our 80 percent target.

Recall is 67.2 percent. This means the model finds about two out of every three items that are really there. So it misses some items, but it rarely invents one.

The mAP at 0.5 is 0.744. This is one overall score, from zero to one, that combines how accurate and how complete the detections are across all three classes.

The chart at the bottom left uses 19 pictures from our third session at the creek, with 361 labelled items, that none of the models were trained on. The score rose from 0.149 for version 7, to 0.204 for version 8, to 0.337 for version 9, and to 0.404 for version 10, about double that of version 8. We should note that these pictures come from the same spot and the same day as the pictures version 10 was trained on, so this shows how the model does at this creek, not at a new site.

On the right are the results on creek pictures only. Precision there drops to 68.9 percent. By class, food containers were the strongest at 0.825, bottles were at 0.590, and bags were the weakest at 0.194.

The main lesson here is that pictures taken at the site itself improved the model more than any public dataset did.

[Next slide]

Slide 12: System and field results

Next, the system as a whole.

On the left are the ten system test cases. Every one of them met its expected result in 10 out of 10 trials.

The average time to process one frame was 275 to 350 milliseconds, which is about a third of a second. That falls in the Excellent band, which is anything below half a second.

At the creek, the unit ran for 24 hours in total over six days, with no crash, no freeze and no overheating.

When an alert was raised, the phone received it within 0.1 to 2 seconds over the local network. Through the cloud, it took 5 seconds or more.

And the counter passed all 19 of its automated tests. It raised the alert once when the count reached the threshold, and not again until the count fell back to 15.

[Next slide]

Slide 13: Field accuracy and evaluation

This slide shows how the system did live at the creek.

The chart on the left shows live classification by day, meaning the share of labels the system got right out of all the labels we checked. Over the three days, it got 295 out of 395 right, or 74.7 percent.

But look at how it changed from day to day. On November 6, the camera was about 13 feet above the water. Classification was 62.5 percent, and the dashboard counted only 21 to 31 percent of the items we counted by eye. The team noted that most of the trash was submerged or half submerged.

On November 7, we lowered the camera to about 9 feet. Classification rose to 73.5 percent, and the dashboard counted 40 to 50 percent of the items.

On November 9, we lowered it again to about 7 feet. Classification reached 85.0 percent, the only day above our 80 percent target, and the dashboard counted 49 to 54 percent of the items. So even at 7 feet, about half of the items were still missed.

As the box at the bottom right says, each time the camera went lower, accuracy went up. We attribute this rise to the lower camera position.

The table on the right is our evaluation under ISO/IEC 25010:2023. Classification at 74.7 percent is rated Fair. Alerting at 100 percent is Excellent. Latency at 0.28 to 0.35 seconds is Excellent. Stability, with 24 hours and no crash, is Excellent. And usability scored 4.40 out of 5, which is also Excellent. That score came from four end users who used the system at the creek: two Estero Rangers, a barangay kagawad and a barangay secretary.

[Next slide]

Slide 14: Conclusions and recommendations

To conclude.

Objective 1 was met. The model reached 84.6 percent precision on the frozen test set, above the 80 percent target.

Objective 2 was met. One Raspberry Pi 5 detects, tracks, counts, alerts and syncs, running on battery, without a separate computer.

Objective 3: the system was rated Excellent for speed, stability and usability. Field classification was Fair at 74.7 percent overall, although it reached 85 percent once the camera was lowered.

And fourth, calibrate on site. Because the model misses more items than it invents, the count runs low. In practice, an alert will tend to come late rather than early, and false alarms are rare. That is why the threshold should be set from a hand count at each site. At our creek, it ended at 20 items.

We have eight recommendations. First, mount the camera low. Seven feet gave us the best accuracy, and units should be placed upstream of culverts, where garbage collects. Second, collect more images of bags, since bags were the weakest class. Third, retrain from the unit's own snapshots. The unit already saves a clean copy of every alert picture, which is ready for labelling. Fourth, test at night and in the rainy season, with an infrared camera and a longer deployment. Fifth, give the unit its own connection, such as an LTE modem, or LoRa radio for sending the alerts and counts. Sixth, connect more cameras to one unit, and use wireless cameras. Seventh, reach longer distances with a higher-quality or zoomable camera. And eighth, add data analytics, to show trends from the stored alerts and counts.

In short, Garbage-Guard is a workable early-warning tool for barangays. It replaces a periodic walk along the bank with a continuous count and a dated record of every build-up. In doing so, it supports UN Sustainable Development Goals 6, 9 and 11: clean water and sanitation; industry, innovation and infrastructure; and sustainable cities and communities.

[Next slide]

Slide 15: Thank you

That ends our presentation. On behalf of the group, thank you to the panel and to our adviser for your time and guidance. We are now ready for your questions.